UADE

TPO PROGRAMACIÓN III - PRIMERA ENTREGA

Propuesta seleccionada: Propuesta 1 — Red Social Universitaria para la Colaboración

Grupo Viedma:

Luz Barrientos - 1193268

German Rocha Sanchez - 1195326

Martina Córdoba - 1152500

Materia: Programación III

Profesor: Leon Maria Angela

Fecha de entrega: 23/10/2026

TPO PROGRAMACIÓN III - PRIMERA ENTREGA 1
1. Introducción 3
2. Objetivo del proyecto 3
3. Tecnologías utilizadas 3
4. Funcionalidad 1 — Organización del muro de publicaciones 3
5. Funcionalidad 2 — Cadena de difusión más rápida 5
6. Funcionalidad 3 — Selección de iniciativas para financiar 8
7. Integración de funcionalidades 10
8. Funcionalidades seleccionadas para la segunda entrega 11
9. Organización del trabajo grupal 12
10. Conclusión 12

1. Introducción

El presente trabajo práctico consiste en el desarrollo de una aplicación en Java que simula algunas funcionalidades de una red social universitaria orientada a la colaboración entre estudiantes.

La plataforma permite organizar publicaciones, determinar cadenas de difusión entre usuarios y seleccionar iniciativas universitarias para financiar con un presupuesto limitado.

Para resolver estos problemas se aplican distintas estrategias de diseño y análisis de algoritmos estudiadas durante la materia.

En esta primera etapa se implementan las tres funcionalidades obligatorias de la propuesta seleccionada, utilizando Merge Sort, Dijkstra y Programación Dinámica mediante el problema de la Mochila 0/1.

2. Objetivo del proyecto

El objetivo principal es desarrollar una aplicación que permita aplicar algoritmos adecuados a distintos problemas, evaluando su funcionamiento, correctitud y eficiencia.

Los objetivos específicos son:

- Implementar un algoritmo eficiente de ordenamiento de publicaciones.
- Encontrar la cadena de difusión de menor tiempo entre dos usuarios.
- Seleccionar iniciativas que maximicen el impacto total sin superar un presupuesto.
- Analizar y justificar las estrategias algorítmicas utilizadas.
- Verificar los resultados mediante diferentes casos de prueba.
- Desarrollar una solución organizada, modular y comprensible en Java.

3. Tecnologías utilizadas

El proyecto se desarrolla utilizando:

- Lenguaje de programación Java 21.
- Entorno de desarrollo IntelliJ IDEA.
- Git y GitHub para el control de versiones y trabajo colaborativo.
- Aplicación de consola para ejecutar y visualizar los resultados.

El código se organiza en paquetes que separan los modelos de datos, los algoritmos, los datos de ejemplo, las pruebas y la ejecución principal.

4. Funcionalidad 1 — Organización del muro de publicaciones

4.1. Descripción del problema

La plataforma administra publicaciones que contienen un identificador, autor, fecha y hora, cantidad de reacciones y cantidad de comentarios.

Se requiere ordenar las publicaciones de dos maneras:

1. Desde la más reciente hasta la más antigua.
2. Desde la más relevante hasta la menos relevante.

Para este desarrollo se considera como relevancia la suma de reacciones y comentarios.

Para este desarrollo se define la relevancia como la suma de reacciones y comentarios, adoptando un criterio uniforme para comparar las publicaciones.

En caso de empate, se prioriza la publicación más reciente y posteriormente el identificador menor.

4.2. Algoritmo seleccionado

Se seleccionó Merge Sort, perteneciente a la estrategia Divide y Conquista.

El algoritmo divide el arreglo de publicaciones en dos mitades, ordena cada mitad recursivamente y combina los resultados para obtener un arreglo ordenado.

La implementación se encuentra en la clase MergeSortPublicaciones.java, dentro del paquete redsocial.algoritmos.

4.3. Justificación de la elección

Merge Sort presenta una complejidad temporal O(n log n) en el mejor, promedio y peor caso.

Esta característica lo convierte en una alternativa adecuada para trabajar con cantidades crecientes de publicaciones.

Además, permite implementar criterios personalizados de comparación sin utilizar métodos de ordenamiento provistos por Java.

4.4. Alternativas consideradas

Se consideró utilizar Bubble Sort, pero se descartó debido a su complejidad $O(n^2)$ en el peor caso, que resulta menos conveniente para cantidades grandes de datos.

También se consideró Quick Sort, que tiene complejidad promedio O(n log n), pero puede alcanzar $O(n^2)$ en su peor caso.

Por este motivo se eligió Merge Sort, que mantiene un rendimiento O(n log n) independientemente del orden inicial de los datos.

4.5. Complejidad

Tiempo: O(n log n).

Espacio auxiliar: O(n).

Donde n representa la cantidad de publicaciones.

4.6. Casos de prueba

Las pruebas se implementaron en la clase PruebasMergeSort.java.

CP01 — Ordenamiento por fecha

- Resultado esperado: IDs 2, 4, 1, 5.
- Resultado obtenido: coincide con lo esperado.
- Estado: Correcto.

CP02 — Ordenamiento por relevancia

- Resultado esperado: IDs 1, 2, 4, 5.
- Resultado obtenido: coincide con lo esperado.
- Estado: Correcto.

CP03 — Empate por relevancia y fecha

- Resultado esperado: se prioriza el identificador menor, obteniendo los IDs 3, 8.
- Resultado obtenido: coincide con lo esperado.
- Estado: Correcto.

CP04 — Arreglo vacío

- Resultado esperado: arreglo vacío.
- Resultado obtenido: coincide con lo esperado.
- Estado: Correcto.

CP05 — Una sola publicación

- Resultado esperado: se conserva la publicación con ID 1.
- Resultado obtenido: coincide con lo esperado.
- Estado: Correcto.

CP06 — Conservación del arreglo original

- Resultado esperado: se mantiene el orden original de los IDs 1, 2, 4, 5.
- Resultado obtenido: coincide con lo esperado.
- Estado: Correcto.

Resultado general: 6 pruebas correctas y 0 fallidas.

5. Funcionalidad 2 — Cadena de difusión más rápida

5.1. Descripción del problema

La plataforma dispone de una red de contactos entre usuarios.

Cada vínculo tiene un sentido de transmisión y un tiempo estimado positivo, expresado en minutos.

Se requiere encontrar la cadena que permita transmitir una convocatoria desde un usuario de origen hasta uno de destino en el menor tiempo posible.

El sistema debe mostrar el recorrido, el tiempo total y las principales decisiones realizadas por el algoritmo.

5.2. Representación de los datos

La red se representa mediante un grafo dirigido y ponderado.

- Los vértices representan usuarios.
- Las aristas representan vínculos de difusión.
- Los pesos representan los minutos necesarios para transmitir la convocatoria.

Se utiliza una lista de adyacencia para almacenar las conexiones de cada usuario.

5.3. Algoritmo seleccionado

Se seleccionó el algoritmo de Dijkstra, que permite encontrar caminos mínimos en grafos con pesos no negativos.

El algoritmo comienza asignando distancia cero al usuario de origen y distancia infinita a los demás.

Posteriormente selecciona el usuario con menor distancia acumulada y actualiza las distancias de sus vecinos cuando encuentra un camino más corto.

La implementación se encuentra en la clase Dijkstra.java, dentro del paquete redsocial.algoritmos.

5.4. Justificación de la elección

Dijkstra resulta adecuado porque todos los tiempos de transmisión son positivos y se necesita encontrar el camino de menor tiempo desde un origen determinado.

La utilización de una cola de prioridad permite seleccionar eficientemente el siguiente usuario que debe procesarse.

5.5. Alternativas consideradas

Se consideró BFS, pero se descartó porque encuentra caminos con la menor cantidad de aristas y no necesariamente con el menor tiempo cuando los pesos son diferentes.

También se consideró Floyd-Warshall, que permite calcular caminos mínimos entre todos los pares de vértices. Sin embargo, para esta funcionalidad se necesita resolver una consulta entre un origen y un destino, por lo que Dijkstra resulta más apropiado.

5.6. Complejidad

Tiempo: O((V + E) log V), utilizando una lista de adyacencia y una cola de prioridad.

Espacio auxiliar: O(V + E).

Donde V representa la cantidad de usuarios y E la cantidad de vínculos.

5.7. Casos de prueba

Las pruebas se implementaron en la clase PruebasDijkstra.java.

CP13 — Camino mínimo de Ana a Eva

- Resultado esperado: Ana → Caro → Beto → Dani → Eva.
- Tiempo esperado: 11 minutos.
- Resultado obtenido: coincide con lo esperado.
- Estado: Correcto.

CP14 — Destino inalcanzable

- Consulta: Eva → Ana.
- Resultado esperado: no existe cadena de difusión.
- Resultado obtenido: coincide con lo esperado.
- Estado: Correcto.

CP15 — Origen igual al destino

- Consulta: Ana → Ana.
- Resultado esperado: Ana, con tiempo total de 0 minutos.
- Resultado obtenido: coincide con lo esperado.
- Estado: Correcto.

CP16 — Usuario inexistente

- Resultado esperado: se rechaza la entrada mediante una excepción controlada.
- Resultado obtenido: coincide con lo esperado.
- Estado: Correcto.

CP17 — Camino indirecto más rápido

- Consulta: Ana → Beto.
- Resultado esperado: Ana → Caro → Beto, con tiempo total de 3 minutos.
- Resultado obtenido: coincide con lo esperado.
- Estado: Correcto.

CP18 — Registro de decisiones

- Resultado esperado: se genera una lista de decisiones no vacía.
- Resultado obtenido: coincide con lo esperado.
- Estado: Correcto.

Resultado general: 6 pruebas correctas y 0 fallidas.

6. Funcionalidad 3 — Selección de iniciativas para financiar

6.1. Descripción del problema

La universidad cuenta con un presupuesto limitado para financiar iniciativas.

Cada iniciativa tiene un identificador, un costo entero positivo y un puntaje de impacto entero no negativo.

Se requiere seleccionar un conjunto de iniciativas que maximice el impacto total sin superar el presupuesto disponible.

Cada iniciativa puede seleccionarse una sola vez y debe financiarse completamente.

6.2. Algoritmo seleccionado

Se seleccionó Programación Dinámica, mediante el problema conocido como Mochila 0/1.

El algoritmo utiliza una matriz para almacenar el máximo impacto alcanzable considerando una cantidad determinada de iniciativas y un presupuesto disponible.

Para cada iniciativa se evalúan dos posibilidades:

1. No financiar y conservar el impacto anterior.
2. Financiar, siempre que el presupuesto lo permita, y sumar su impacto al mejor resultado compatible.

Se selecciona la alternativa que produce el mayor impacto.

La implementación se encuentra en la clase Mochila.java, dentro del paquete redsocial.algoritmos.

6.3. Justificación de la elección

La Programación Dinámica permite reutilizar resultados de subproblemas y evita evaluar repetidamente las mismas combinaciones.

Esto resulta conveniente porque el objetivo es encontrar una solución óptima y no simplemente una selección válida.

6.4. Alternativas consideradas

Se consideró utilizar un algoritmo Greedy, seleccionando las iniciativas según su impacto o su relación impacto/costo.

Sin embargo, esta estrategia no garantiza encontrar siempre la combinación de mayor impacto para el problema de Mochila 0/1.

También se consideró evaluar todas las combinaciones mediante fuerza bruta, pero su complejidad exponencial $O(2^n)$ resulta menos conveniente cuando aumenta la cantidad de iniciativas.

Por estas razones se eligió Programación Dinámica.

6.5. Complejidad

Tiempo: $O(n \times P)$.

Espacio auxiliar: $O(n \times P)$.

Donde n representa la cantidad de iniciativas y P el presupuesto disponible.

Se trata de una complejidad pseudopolinomial, ya que depende del valor numérico del presupuesto.

6.6. Casos de prueba

Para las pruebas se consideran las siguientes iniciativas:

- Iniciativa 1: costo 4, impacto 7.
- Iniciativa 2: costo 5, impacto 9.
- Iniciativa 3: costo 3, impacto 6.
- Iniciativa 4: costo 2, impacto 4.

Las pruebas se implementaron en la clase PruebasMochila.java.

CP07 — Presupuesto 9

- Resultado esperado: iniciativas 1, 3 y 4.
- Costo total: 9.
- Impacto total: 17.
- Presupuesto restante: 0.

Resultado obtenido: coincide con lo esperado.

- Estado: Correcto.

CP08 — Presupuesto 2

- Resultado esperado: iniciativa 4.
- Costo total: 2.
- Impacto total: 4.
- Resultado obtenido: coincide con lo esperado.
- Estado: Correcto.

CP09 — Presupuesto 0

- Resultado esperado: ninguna iniciativa seleccionada.
- Impacto total: 0.
- Resultado obtenido: coincide con lo esperado.
- Estado: Correcto.

CP10 — Presupuesto suficiente

- Presupuesto disponible: 14.
- Resultado esperado: selección de las cuatro iniciativas.
- Costo total: 14.
- Impacto total: 26.
- Resultado obtenido: coincide con lo esperado.
- Estado: Correcto.

CP11 — Lista de iniciativas vacía

- Resultado esperado: ninguna iniciativa seleccionada.
- Costo total: 0.
- Impacto total: 0.
- Resultado obtenido: coincide con lo esperado.
- Estado: Correcto.

CP12 — Presupuesto negativo

- Resultado esperado: se rechaza el presupuesto negativo mediante una excepción controlada.
- Resultado obtenido: coincide con lo esperado.
- Estado: Correcto.

Resultado general: 6 pruebas correctas y 0 fallidas.

7. Integración de funcionalidades

Las tres funcionalidades se integran en una aplicación de consola mediante un menú principal.

El usuario puede seleccionar el ordenamiento de publicaciones, buscar una cadena de difusión o calcular la selección de iniciativas.

La implementación separa los datos, los algoritmos y la ejecución del programa para facilitar su mantenimiento, comprensión y posterior ampliación.

El proyecto se encuentra organizado en los siguientes paquetes:

- redsocial.modelo: contiene las clases Publicacion, Vinculo e Iniciativa.
- redsocial.algoritmos: contiene las implementaciones de Merge Sort, Dijkstra y Mochila 0/1.
- redsocial.datos: contiene la clase DatosEjemplo, utilizada para cargar los datos de prueba.
- redsocial.pruebas: contiene las clases de pruebas de cada algoritmo y su ejecución conjunta.

La clase Main.java permite acceder a las funcionalidades desde el menú principal.

La clase EjecutarPruebas.java permite ejecutar automáticamente los 18 casos de prueba.

Se ejecutaron las 18 pruebas correspondientes a las tres funcionalidades, obteniendo 18 resultados correctos y 0 fallidos.

La documentación detallada de las pruebas se encuentra en el archivo CASOS_DE_PRUEBA, dentro de la carpeta pruebas del proyecto.

8. Funcionalidades seleccionadas para la segunda entrega

Para la segunda etapa se seleccionaron las siguientes funcionalidades:

A3 — Selección de actividades compatibles

Se implementará un algoritmo que seleccione actividades que no se superpongan, maximizando el interés total.

La estrategia prevista es Programación Dinámica, mediante selección ponderada de intervalos.

B1 — Formación de equipos con restricciones

Se implementará un algoritmo para encontrar todos los equipos válidos que cumplan con el tamaño requerido, las habilidades necesarias, los integrantes obligatorios y las incompatibilidades.

La estrategia prevista es Backtracking.

B3 — Cadenas alternativas de difusión

Se implementará un algoritmo que encuentre todas las cadenas de difusión válidas entre dos usuarios, respetando límites de tiempo, cantidad de intermediarios y usuarios excluidos.

La estrategia prevista es Backtracking, utilizando la red de contactos desarrollada durante la primera etapa.

9. Organización del trabajo grupal

El proyecto se desarrolla de manera colaborativa entre tres integrantes:

- Luz Barrientos.
- Germán Rocha Sánchez.
- Martina Córdoba.

Para organizar el desarrollo se utiliza un repositorio Git compartido, que permite registrar los cambios realizados y mantener un historial de las contribuciones.

https://github.com/luzbarrientos/TPO-Viedma-Red-Social

Durante la primera etapa se realizaron las siguientes tareas:

- Implementación del modelo de publicaciones.
- Desarrollo del algoritmo Merge Sort.
- Implementación del modelo de iniciativas.
- Desarrollo del algoritmo de Mochila 0/1.
- Implementación del modelo de vínculos entre usuarios.
- Desarrollo del algoritmo de Dijkstra.
- Preparación de los datos de ejemplo.
- Integración de las funcionalidades mediante el menú principal.
- Implementación de 18 casos de prueba automatizados.
- Documentación y revisión de las funcionalidades desarrolladas.

Las tareas se organizaron de acuerdo con las responsabilidades asumidas por los integrantes durante el desarrollo del proyecto.

La participación individual se registra mediante el historial de cambios del repositorio Git, que permite identificar los aportes realizados por cada integrante.

10. Conclusión

Durante la primera etapa del trabajo se desarrollaron tres funcionalidades que permiten aplicar diferentes estrategias de diseño de algoritmos.

Merge Sort permite ordenar publicaciones eficientemente mediante Divide y Conquista.

Dijkstra permite encontrar cadenas de difusión de tiempo mínimo en una red de usuarios.

La Programación Dinámica permite seleccionar iniciativas para maximizar el impacto total respetando un presupuesto limitado.

La combinación de estas estrategias permite resolver problemas diferentes utilizando técnicas adecuadas para cada situación.

Además, se implementaron y ejecutaron 18 casos de prueba automatizados, distribuidos entre las tres funcionalidades desarrolladas.

Los resultados obtenidos fueron satisfactorios, con 18 pruebas correctas y ninguna fallida.

Estos resultados permiten verificar el funcionamiento de los algoritmos en los escenarios evaluados y continuar con el desarrollo de la segunda etapa.

La segunda etapa ampliará el sistema mediante la selección de actividades compatibles, la formación de equipos y la búsqueda de cadenas alternativas de difusión.